

```
In [2]: import numpy as np
```

Array creation + inspection (type, dtype, ndim, size, shape)

```
In [3]: arr1 = np.array([15, 16, 15, 17, 16])
print(type(arr1))
print(arr1.dtype)
print(arr1.ndim)
print(arr1.size)
print(arr1.shape)
```

```
<class 'numpy.ndarray'>
int64
1
5
(5,)
```

```
In [6]: arr2 = np.array([[36.5, 37.2], [36.8, 38.1], [37.0, 36.9]])
print(type(arr2))
print(arr2.dtype)
print(arr2.ndim)
print(arr2.size)
print(arr2.shape)
```

```
<class 'numpy.ndarray'>
float64
2
6
(3, 2)
```

```
In [7]: arr3 = np.array([[[50, 75.5], [60, 80]], [[45.5, 70], [55, 90.5]]])
print(type(arr3))
print(arr3.dtype)
print(arr3.ndim)
print(arr3.size)
print(arr3.shape)
```

```
<class 'numpy.ndarray'>
float64
3
8
(2, 2, 2)
```

2) reshape() — normal, -1 auto-dim, 3D

```
In [8]: a = np.array([120, 150, 130, 160, 140, 170, 155, 165, 145, 175, 135, 180])
print(a.reshape(3, 4))
```

```
[[120 150 130 160]
 [140 170 155 165]
 [145 175 135 180]]
```

```
In [9]: b = np.array([56, 78, 90, 45, 67, 89, 34, 72, 88, 91, 60, 77])
print(b.reshape(-1, 3))
```

```
[[56 78 90]
 [45 67 89]
 [34 72 88]
 [91 60 77]]
```

```
In [10]: c = np.array([30, 32, 31, 29, 33, 30, 25, 27, 26, 24, 28, 27])
print(c.reshape(2, 2, 3))
```

```
[[[30 32 31]
  [29 33 30]]

 [[25 27 26]
  [24 28 27]]]
```

3) std() and var()

```
In [11]: d1 = np.array([88, 90, 87, 91, 89])
print(d1.std())
print(d1.var())
```

```
1.4142135623730951
2.0
```

```
In [12]: d2 = np.array([15000, 90000, 45000, 200000, 30000])
print(d2.std())
print(d2.var())
```

```
66887.96603276258
4474000000.0
```

```
In [13]: d3 = np.array([10.01, 10.00, 10.02, 10.01, 10.00])
print(d3.std())
print(d3.var())
```

```
0.007483314773547724
5.599999999999762e-05
```

4) eye() and zeros()

```
In [14]: print(np.eye(2))
print(np.eye(3))
print(np.eye(5))
```

```
[[1. 0.]
 [0. 1.]]
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
[[1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 1.]]
```

```
In [15]: print(np.zeros(5))
```

```
[0. 0. 0. 0. 0.]
```

```
In [16]: print(np.zeros((3, 4)))
```

```
[[0. 0. 0. 0.]  
 [0. 0. 0. 0.]  
 [0. 0. 0. 0.]]
```

5) copy() — independent from original

```
In [17]: orig = np.array([100, 250, 300, 450])  
        copied = orig.copy()  
        orig[0] = 999  
        print(copied)
```

```
[100 250 300 450]
```

```
In [18]: orig2 = np.array([56, 78, 90, 45, 67])  
        copied2 = orig2.copy()  
        orig2[0] = 100  
        print(copied2)
```

```
[56 78 90 45 67]
```

```
In [19]: orig3 = np.array([25.5, 26.0, 24.8, 27.3])  
        copied3 = orig3.copy()  
        orig3[0] = 0.0  
        print(copied3)
```

```
[25.5 26.  24.8 27.3]
```

6) sum(axis=0) and sum(axis=1)

```
In [20]: mat1 = np.array([[100, 150, 200], [80, 90, 120]])  
        print(mat1.sum(axis=0))  
        print(mat1.sum(axis=1))
```

```
[180 240 320]  
[450 290]
```

```
In [21]: mat2 = np.array([[78, 65, 90, 82], [55, 60, 70, 65], [88, 92, 85, 95]])  
        print(mat2.sum(axis=0))  
        print(mat2.sum(axis=1))
```

```
[221 217 245 242]  
[315 250 360]
```

```
In [22]: mat3 = np.array([[12, 8], [20, 15], [5, 3], [18, 22]])  
        print(mat3.sum(axis=0))  
        print(mat3.sum(axis=1))
```

```
[55 48]  
[20 35  8 40]
```

7) ravel() vs flatten() — view vs copy

ravel() — 3 implementations

```
In [23]: # Implementation 1: Classroom seating numbers (2x2 grid)
base1 = np.array([[1, 2], [3, 4]])
r1 = base1.ravel()
r1[0] = 999
print(base1)    # changed – ravel() is a view, linked to original

[[999  2]
 [ 3  4]]
```

```
In [24]: # Implementation 2: Product price grid (2 products x 2 warehouses)
base2 = np.array([[500, 750], [300, 620]])
r2 = base2.ravel()
r2[0] = 9999
print(base2)    # changed – same memory as original

[[9999  750]
 [ 300  620]]
```

```
In [25]: # Implementation 3: Temperature readings (2 days x 2 time slots)
base3 = np.array([[22.5, 28.0], [24.1, 29.3]])
r3 = base3.ravel()
r3[0] = 0.0
print(base3)    # changed – ravel() reflects back on original

[[ 0.  28. ]
 [24.1 29.3]]
```

flatten() — 3 implementations

```
In [26]: base4 = np.array([[1, 2], [3, 4]])
f1 = base4.flatten()
f1[0] = 111
print(base4)

[[1 2]
 [3 4]]
```

```
In [27]: scores = np.array([[85, 90], [78, 92]])    # original marks, safe

temp = scores.flatten()
temp[0] = 0

print(scores)
print(temp)

[[85 90]
 [78 92]]
[ 0 90 78 92]
```

```
In [28]: marks = np.array([[85, 78], [92, 88], [60, 70]])

all_marks = marks.flatten()
highest = np.sort(all_marks)[-1]
```

```
print(highest)
print(marks)
```

```
92
[[85 78]
 [92 88]
 [60 70]]
```

8) Boolean masking with & (AND)

```
In [29]: data1 = np.array([45, 55, 60, 90, 35, 72, 88, 50])
mask1 = (data1 > 50) & (data1 < 80)
print(data1[mask1])
```

```
[55 60 72]
```

```
In [30]: data2 = np.array([4, 15, 22, 7, 30, 9, 16])
mask2 = (data2 % 2 == 0) & (data2 > 10)
print(data2[mask2])
```

```
[22 30 16]
```

```
In [31]: data3 = np.array([35, 42, 58, 95, 61, 33, 78])
mask3 = (data3 >= 33) & (data3 != 95)

print(data3[mask3])
```

```
[35 42 58 61 33 78]
```

9) transpose()

```
In [32]: t = np.array([[1, 2, 3], [4, 5, 6]])

print(t.transpose())
```

```
[[1 4]
 [2 5]
 [3 6]]
```

```
In [33]: t = np.array([
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12]
])

print(t.transpose())
```

```
[[ 1  5  9]
 [ 2  6 10]
 [ 3  7 11]
 [ 4  8 12]]
```

```
In [34]: temperature = np.array([
    [32, 28, 25],
    [35, 30, 27],
    [31, 29, 24],
    [33, 31, 26]
])
```

```
print(temperature.transpose())
```

```
[[32 35 31 33]
 [28 30 29 31]
 [25 27 24 26]]
```

10)broadcasting on arrays

```
In [35]: roll_numbers = np.array([101, 102, 103, 104, 105])
print(roll_numbers % 2)
```

```
[1 0 1 0 1]
```

```
In [36]: marks = np.array([65, 72, 80, 68, 90])
bonus = 5
print(marks + bonus)
```

```
[70 77 85 73 95]
```

```
In [37]: prices = np.array([500, 800, 1000, 1500, 2000])
discount = 0.10
print(prices - (prices * discount))
```

```
[ 450.  720.  900. 1350. 1800.]
```

11) List repetition vs array broadcasting

```
In [38]: py_list = [1, 2, 3]
print(py_list * 3)

np_arr = np.array([1, 2, 3])
print(np_arr * 3)
```

```
[1, 2, 3, 1, 2, 3, 1, 2, 3]
[3 6 9]
```

```
In [39]: py_list = ["Aman", "Riya", "Siya"]
print(py_list * 2)

np_arr = np.array([10, 20, 30])
print(np_arr * 2)

['Aman', 'Riya', 'Siya', 'Aman', 'Riya', 'Siya']
[20 40 60]
```

```
In [40]: py_list = [2, 5, 3]
print(py_list * 3)

np_arr = np.array([2, 5, 3])
print(np_arr * 3)
```

```
[2, 5, 3, 2, 5, 3, 2, 5, 3]
[ 6 15  9]
```

12) Boolean masking with ~ (NOT / inverse)

```
In [41]: marks = np.array([35, 72, 28, 90, 45, 20])

mask = (marks >= 40)

print(marks[mask])    # Passed
print(marks[~mask])   # Failed
```

```
[72 90 45]
[35 28 20]
```

```
In [42]: prices = np.array([500, 1200, 800, 2500, 1500, 300])

mask = (prices >= 1000)

print(prices[mask])    # ₹1000 or more
print(prices[~mask])   # Below ₹1000
```

```
[1200 2500 1500]
[500 800 300]
```

```
In [43]: temperature = np.array([25, 32, 38, 29, 41, 27])

mask = (temperature >= 35)

print(temperature[mask])    # Hot
print(temperature[~mask])   # Not hot
```

```
[38 41]
[25 32 29 27]
```

13) Range masking with & (between two values)

```
In [44]: numbers = np.array([10, 15, 20, 25, 30, 35, 40])

mask = (numbers % 2 == 0) & (numbers % 5 == 0)

print(numbers[mask])
```

```
[10 20 30 40]
```

```
In [45]: ages = np.array([18, 20, 18, 22, 21, 18])

mask = (ages == 18) & (ages % 2 == 0)

print(ages[mask])
```

```
[18 18 18]
```

```
In [46]: numbers = np.array([12, 15, 20, 30, 45, 50, 60])
```

```
mask = (numbers % 3 == 0) & (numbers % 5 == 0)

print(numbers[mask])
```

```
[15 30 45 60]
```

14) np.select() — single condition

```
In [47]: age = np.array([12, 18, 15, 21, 16])

print(np.select([age >= 18], ["Adult"], default="Minor"))

['Minor' 'Adult' 'Minor' 'Adult' 'Minor']
```

```
In [48]: attendance = np.array([65, 80, 75, 90, 55])

print(np.select([attendance >= 75], ["Eligible"], default="Not Eligible"))

['Not Eligible' 'Eligible' 'Eligible' 'Eligible' 'Not Eligible']
```

```
In [49]: numbers = np.array([10, -5, 8, -2, 15])

print(np.select([numbers >= 0], ["Positive"], default="Negative"))

['Positive' 'Negative' 'Positive' 'Negative' 'Positive']
```

15) np.select() — multiple conditions with default

```
In [50]: marks = np.array([95, 82, 76, 68, 55, 43, 31, 89, 72, 60, 97, 25])

cond = [
    (marks >= 80) & (marks <= 100),
    (marks >= 60) & (marks < 80)
]

choice = ["A Grade", "B Grade"]

print(np.select(cond, choice, default="C Grade"))

['A Grade' 'A Grade' 'B Grade' 'B Grade' 'C Grade' 'C Grade' 'C Grade'
 'A Grade' 'B Grade' 'B Grade' 'A Grade' 'C Grade']
```

```
In [51]: experience = np.array([0, 1, 2, 3, 4, 5, 6, 8, 10, 2, 7, 12])

cond = [
    (experience >= 3) & (experience <= 5),
    (experience > 5) & (experience <= 10)
]

choice = ["Mid-Level", "Senior"]

print(np.select(cond, choice, default="Fresher"))

['Fresher' 'Fresher' 'Fresher' 'Mid-Level' 'Mid-Level' 'Mid-Level'
 'Senior' 'Senior' 'Senior' 'Fresher' 'Senior' 'Fresher']
```

```
In [52]: orders = np.array([
    350, 600, 850, 1200, 1500, 1800,
    2200, 750, 950, 1300, 2500, 450
])

cond = [
    (orders >= 500) & (orders <= 999),
    (orders >= 1000) & (orders <= 1999)
]

choice = ["5% Discount", "10% Discount"]

print(np.select(cond, choice, default="No Discount"))

['No Discount' '5% Discount' '5% Discount' '10% Discount' '10% Discount'
'10% Discount' 'No Discount' '5% Discount' '5% Discount' '10% Discount'
'No Discount' 'No Discount']

import numpy as np
```

===== NUMPY IMPLEMENTATIONS =====

1. np.array()

List se NumPy array banata hai.

2. type()

Object ka data type check karta hai.

3. .dtype

Array ke elements ka datatype batata hai.

4. .ndim

Array ke dimensions (1D/2D/3D) ki ginti batata hai.

5. .size

Array mein total elements ki ginti batata hai.

6. .sum()

Sabhi elements ka total nikalta hai.

7. .shape

Array ka structure batata hai, jaise rows aur columns.

8. .reshape()

Array ka shape change karta hai, data same rehta hai.

9. np.eye()

Identity matrix banata hai.

10. np.zeros()

Zeros se filled array banata hai.

11. .std()

Standard deviation nikalta hai, yani data kitna spread hai.

12. .var()

Variance nikalta hai, yani data ke spread ka measure.

13. .copy()

Original se independent array ki copy banata hai.

14. .sum(axis=0) / .sum(axis=1)

axis=0 column-wise aur axis=1 row-wise sum karta hai.

15. .ravel()

Array ko 1D karta hai aur generally view return karta hai.

16. .flatten()

Array ko 1D karta hai aur independent copy return karta hai.

17. Boolean Masking (>, <, >=, <=, &, ~)

Condition ke basis par array ke elements filter karta hai.

18. .transpose()

Array ki rows aur columns swap karta hai.

19. Broadcasting ($\%$, $+$, $-$, $*$, $/$)

Array ke elements par element-wise operation apply karta hai.

20. np.where()

Condition ke True/False result ke basis par values choose karta hai.

21. np.select()

Multiple conditions ke basis par different values choose karta hai.

In []:

```
In [1]: import pandas as pd
```

```
In [4]: import pandas as pd
```

```
netflix = pd.read_csv("netflix_titles (2).csv")
```

```
print(netflix.head())
```

	show_id	type	title	director	\
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	
1	s2	TV Show	Blood & Water	NaN	
2	s3	TV Show	Ganglands	Julien Leclercq	
3	s4	TV Show	Jailbirds New Orleans	NaN	
4	s5	TV Show	Kota Factory	NaN	

	cast	country	\
0	NaN	United States	
1	Ama Qamata, Khosi Ngema, Gail Mablane, Thaban...	South Africa	
2	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	
3	NaN	NaN	
4	Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...	India	

	date_added	release_year	rating	duration	\
0	September 25, 2021	2020	PG-13	90 min	
1	September 24, 2021	2021	TV-MA	2 Seasons	
2	September 24, 2021	2021	TV-MA	1 Season	
3	September 24, 2021	2021	TV-MA	1 Season	
4	September 24, 2021	2021	TV-MA	2 Seasons	

	listed_in	\
0	Documentaries	
1	International TV Shows, TV Dramas, TV Mysteries	
2	Crime TV Shows, International TV Shows, TV Act...	
3	Docuseries, Reality TV	
4	International TV Shows, Romantic TV Shows, TV ...	

	description
0	As her father nears the end of his life, filmm...
1	After crossing paths at a party, a Cape Town t...
2	To protect his family from a powerful drug lor...
3	Feuds, flirtations and toilet talk go down amo...
4	In a city of coaching centers known to train I...

```
In [5]: import pandas as pd
```

```
co2 = pd.read_csv("CO2 Emissions_Canada.csv")
```

```
print(co2.head())
```

	Make	Model	Vehicle Class	Engine Size(L)	Cylinders	Transmission	\
0	ACURA	ILX	COMPACT	2.0	4	AS5	
1	ACURA	ILX	COMPACT	2.4	4	M6	
2	ACURA	ILX HYBRID	COMPACT	1.5	4	AV7	
3	ACURA	MDX 4WD	SUV - SMALL	3.5	6	AS6	
4	ACURA	RDX AWD	SUV - SMALL	3.5	6	AS6	

	Fuel Type	Fuel Consumption City (L/100 km)	\
0	Z	9.9	
1	Z	11.2	
2	Z	6.0	
3	Z	12.7	
4	Z	12.1	

	Fuel Consumption Hwy (L/100 km)	Fuel Consumption Comb (L/100 km)	\
0	6.7	8.5	
1	7.7	9.6	
2	5.8	5.9	
3	9.1	11.1	
4	8.7	10.6	

	Fuel Consumption Comb (mpg)	CO2 Emissions(g/km)
0	33	196
1	29	221
2	48	136
3	25	255
4	27	244

```
In [6]: import pandas as pd

iris = pd.read_csv("Iris.csv")

print(iris.head())
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

Creating a DataFrame from Student Records

```
In [7]: import pandas as pd

data = {
    "Name": ["Siya", "Aman", "Riya", "Karan"],
    "Marks": [85, 92, 78, 88],
    "Attendance": [90, 95, 82, 89]
}

students = pd.DataFrame(data)

print(students)
```

	Name	Marks	Attendance
0	Siya	85	90
1	Aman	92	95
2	Riya	78	82
3	Karan	88	89

```
In [8]: data = {
    "Employee": ["Aman", "Rohit", "Neha", "Priya"],
    "Department": ["IT", "HR", "IT", "Finance"],
    "Salary": [45000, 40000, 52000, 48000]
}

employees = pd.DataFrame(data)

print(employees)
```

	Employee	Department	Salary
0	Aman	IT	45000
1	Rohit	HR	40000
2	Neha	IT	52000
3	Priya	Finance	48000

```
In [9]: data = {
    "Product": ["Laptop", "Mouse", "Keyboard", "Monitor"],
    "Quantity": [15, 40, 25, 12],
    "Price": [55000, 800, 1500, 12000]
}

sales = pd.DataFrame(data)

print(sales)
```

	Product	Quantity	Price
0	Laptop	15	55000
1	Mouse	40	800
2	Keyboard	25	1500
3	Monitor	12	12000

Viewing First 5 Student Records

```
In [10]: print(students.head())
```

	Name	Marks	Attendance
0	Siya	85	90
1	Aman	92	95
2	Riya	78	82
3	Karan	88	89

```
In [12]: # Viewing First 6 Netflix Records

netflix = pd.read_csv("netflix_titles (2).csv")

print(netflix.head(6))
```

	show_id	type	title	director	\
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	
1	s2	TV Show	Blood & Water	NaN	
2	s3	TV Show	Ganglands	Julien Leclercq	
3	s4	TV Show	Jailbirds New Orleans	NaN	
4	s5	TV Show	Kota Factory	NaN	
5	s6	TV Show	Midnight Mass	Mike Flanagan	

	cast	country	\
0	NaN	United States	
1	Ama Qamata, Khosi Ngema, Gail Mabalanane, Thaban...	South Africa	
2	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	
3	NaN	NaN	
4	Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...	India	
5	Kate Siegel, Zach Gilford, Hamish Linklater, H...	NaN	

	date_added	release_year	rating	duration	\
0	September 25, 2021	2020	PG-13	90 min	
1	September 24, 2021	2021	TV-MA	2 Seasons	
2	September 24, 2021	2021	TV-MA	1 Season	
3	September 24, 2021	2021	TV-MA	1 Season	
4	September 24, 2021	2021	TV-MA	2 Seasons	
5	September 24, 2021	2021	TV-MA	1 Season	

	listed_in	\
0	Documentaries	
1	International TV Shows, TV Dramas, TV Mysteries	
2	Crime TV Shows, International TV Shows, TV Act...	
3	Docuseries, Reality TV	
4	International TV Shows, Romantic TV Shows, TV ...	
5	TV Dramas, TV Horror, TV Mysteries	

	description
0	As her father nears the end of his life, filmm...
1	After crossing paths at a party, a Cape Town t...
2	To protect his family from a powerful drug lor...
3	Feuds, flirtations and toilet talk go down amo...
4	In a city of coaching centers known to train I...
5	The arrival of a charismatic young priest brin...

In [13]: *# Viewing First 7 CO2 Emission Records*

```
co2 = pd.read_csv("CO2 Emissions_Canada.csv")

print(co2.head())
```

	Make	Model	Vehicle Class	Engine Size(L)	Cylinders	Transmission	\
0	ACURA	ILX	COMPACT	2.0	4	AS5	
1	ACURA	ILX	COMPACT	2.4	4	M6	
2	ACURA	ILX HYBRID	COMPACT	1.5	4	AV7	
3	ACURA	MDX 4WD	SUV - SMALL	3.5	6	AS6	
4	ACURA	RDX AWD	SUV - SMALL	3.5	6	AS6	

	Fuel Type	Fuel Consumption City (L/100 km)	\
0	Z	9.9	
1	Z	11.2	
2	Z	6.0	
3	Z	12.7	
4	Z	12.1	

	Fuel Consumption Hwy (L/100 km)	Fuel Consumption Comb (L/100 km)	\
0	6.7	8.5	
1	7.7	9.6	
2	5.8	5.9	
3	9.1	11.1	
4	8.7	10.6	

	Fuel Consumption Comb (mpg)	CO2 Emissions(g/km)
0	33	196
1	29	221
2	48	136
3	25	255
4	27	244

Checking Data Information

In [14]: `students.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Name        4 non-null      object
1   Marks       4 non-null      int64
2   Attendance  4 non-null      int64
dtypes: int64(2), object(1)
memory usage: 228.0+ bytes
```

In [15]: `netflix.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   show_id                8807 non-null   object
1   type                   8807 non-null   object
2   title                  8807 non-null   object
3   director               6173 non-null   object
4   cast                   7982 non-null   object
5   country                7976 non-null   object
6   date_added             8797 non-null   object
7   release_year           8807 non-null   int64
8   rating                 8803 non-null   object
9   duration               8804 non-null   object
10  listed_in              8807 non-null   object
11  description             8807 non-null   object
dtypes: int64(1), object(11)
memory usage: 825.8+ KB
```

In [16]: `co2.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7385 entries, 0 to 7384
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Make                                  7385 non-null   object
1   Model                                7385 non-null   object
2   Vehicle Class                         7385 non-null   object
3   Engine Size(L)                        7385 non-null   float64
4   Cylinders                             7385 non-null   int64
5   Transmission                          7385 non-null   object
6   Fuel Type                             7385 non-null   object
7   Fuel Consumption City (L/100 km)      7385 non-null   float64
8   Fuel Consumption Hwy (L/100 km)       7385 non-null   float64
9   Fuel Consumption Comb (L/100 km)      7385 non-null   float64
10  Fuel Consumption Comb (mpg)           7385 non-null   int64
11  CO2 Emissions(g/km)                  7385 non-null   int64
dtypes: float64(4), int64(3), object(5)
memory usage: 692.5+ KB
```

Checking Data Number of Rows and Columns

In [17]: `print(students.shape)`

(4, 3)

In [18]: `print(netflix.shape)`

(8807, 12)

In [19]: `print(co2.shape)`

(7385, 12)

Get Column Names

```
In [20]: print(students.columns)
```

```
Index(['Name', 'Marks', 'Attendance'], dtype='object')
```

```
In [21]: print(netflix.columns)
```

```
Index(['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added',  
      'release_year', 'rating', 'duration', 'listed_in', 'description'],  
      dtype='object')
```

```
In [22]: print(co2.columns)
```

```
Index(['Make', 'Model', 'Vehicle Class', 'Engine Size(L)', 'Cylinders',  
      'Transmission', 'Fuel Type', 'Fuel Consumption City (L/100 km)',  
      'Fuel Consumption Hwy (L/100 km)', 'Fuel Consumption Comb (L/100 km)',  
      'Fuel Consumption Comb (mpg)', 'CO2 Emissions(g/km)'],  
      dtype='object')
```

Find Missing Values

```
In [23]: print(students.isnull())
```

	Name	Marks	Attendance
0	False	False	False
1	False	False	False
2	False	False	False
3	False	False	False

```
In [24]: print(netflix.isnull())
```

	show_id	type	title	director	cast	country	date_added	\
0	False	False	False	False	True	False	False	
1	False	False	False	True	False	False	False	
2	False	False	False	False	False	True	False	
3	False	False	False	True	True	True	False	
4	False	False	False	True	False	False	False	
...	...	...	...	...	...	...	...	
8802	False	False	False	False	False	False	False	
8803	False	False	False	True	True	True	False	
8804	False	False	False	False	False	False	False	
8805	False	False	False	False	False	False	False	
8806	False	False	False	False	False	False	False	

	release_year	rating	duration	listed_in	description
0	False	False	False	False	False
1	False	False	False	False	False
2	False	False	False	False	False
3	False	False	False	False	False
4	False	False	False	False	False
...	...	...	...	...	...
8802	False	False	False	False	False
8803	False	False	False	False	False
8804	False	False	False	False	False
8805	False	False	False	False	False
8806	False	False	False	False	False

```
[8807 rows x 12 columns]
```

```
In [25]: print(co2.isnull())
```

	Make	Model	Vehicle Class	Engine Size(L)	Cylinders	Transmission	\
0	False	False	False	False	False	False	
1	False	False	False	False	False	False	
2	False	False	False	False	False	False	
3	False	False	False	False	False	False	
4	False	False	False	False	False	False	
...	...	...	...	...	...	...	
7380	False	False	False	False	False	False	
7381	False	False	False	False	False	False	
7382	False	False	False	False	False	False	
7383	False	False	False	False	False	False	
7384	False	False	False	False	False	False	

	Fuel Type	Fuel Consumption City (L/100 km)	\
0	False	False	
1	False	False	
2	False	False	
3	False	False	
4	False	False	
...	...	...	
7380	False	False	
7381	False	False	
7382	False	False	
7383	False	False	
7384	False	False	

	Fuel Consumption Hwy (L/100 km)	Fuel Consumption Comb (L/100 km)	\
0	False	False	
1	False	False	
2	False	False	
3	False	False	
4	False	False	
...	...	...	
7380	False	False	
7381	False	False	
7382	False	False	
7383	False	False	
7384	False	False	

	Fuel Consumption Comb (mpg)	CO2 Emissions(g/km)
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False
...	...	...
7380	False	False
7381	False	False
7382	False	False
7383	False	False
7384	False	False

```
[7385 rows x 12 columns]
```

Count Missing Values

```
In [27]: print(students.isnull().sum())
```

```
Name          0
Marks          0
Attendance     0
dtype: int64
```

```
In [28]: print(netflix.isnull().sum())
```

```
show_id        0
type           0
title          0
director       2634
cast           825
country        831
date_added     10
release_year   0
rating         4
duration       3
listed_in      0
description     0
dtype: int64
```

```
In [29]: print(co2.isnull().sum())
```

```
Make          0
Model         0
Vehicle Class 0
Engine Size(L) 0
Cylinders     0
Transmission  0
Fuel Type     0
Fuel Consumption City (L/100 km) 0
Fuel Consumption Hwy (L/100 km) 0
Fuel Consumption Comb (L/100 km) 0
Fuel Consumption Comb (mpg)      0
CO2 Emissions(g/km)              0
dtype: int64
```

Find Duplicate Rows

```
In [30]: student_data = {
    "Name": ["Siya", "Aman", "Riya", "Aman", "Karan"],
    "Marks": [85, 92, 78, 92, 88],
    "Attendance": [90, 95, 82, 95, 89]
}

students2 = pd.DataFrame(student_data)

print(students2.duplicated())
```

```
0    False
1    False
2    False
3     True
4    False
dtype: bool
```

```
In [31]: product_data = {
    "Product": ["Laptop", "Mouse", "Keyboard", "Mouse", "Monitor"],
    "Price": [55000, 800, 1500, 800, 12000],
    "Stock": [10, 50, 30, 50, 15]
}

products = pd.DataFrame(product_data)

print(products.duplicated())

0    False
1    False
2    False
3     True
4    False
dtype: bool
```

```
In [32]: employee_data = {
    "Employee_ID": [101, 102, 103, 102, 104],
    "Name": ["Aman", "Riya", "Karan", "Riya", "Neha"],
    "Department": ["IT", "HR", "Finance", "HR", "IT"]
}

employees2 = pd.DataFrame(employee_data)

print(employees2.duplicated())

0    False
1    False
2    False
3     True
4    False
dtype: bool
```

Count Duplicate Rows

```
In [33]: student_data = {
    "Name": ["Siya", "Aman", "Riya", "Aman", "Karan"],
    "Marks": [85, 92, 78, 92, 88],
    "Attendance": [90, 95, 82, 95, 89]
}

students2 = pd.DataFrame(student_data)

print(students2.duplicated().sum())

1
```

```
In [37]: # Example 2: Counting Duplicate Product Records

product_data = {
    "Product": ["Laptop", "Mouse", "Keyboard", "Mouse", "Monitor"],
    "Price": [55000, 800, 1500, 800, 12000],
    "Stock": [10, 50, 30, 50, 15]
}

products = pd.DataFrame(product_data)
```

```
print(products.duplicated().sum())
```

1

```
In [38]: employee_data = {
    "Employee_ID": [101, 102, 103, 102, 104],
    "Name": ["Aman", "Riya", "Karan", "Riya", "Neha"],
    "Department": ["IT", "HR", "Finance", "HR", "IT"]
}

employees2 = pd.DataFrame(employee_data)

print(employees2.duplicated().sum())
```

1

Statistical Summary

```
In [39]: print(students.describe())
```

	Marks	Attendance
count	4.000000	4.000000
mean	85.750000	89.000000
std	5.909033	5.354126
min	78.000000	82.000000
25%	83.250000	87.250000
50%	86.500000	89.500000
75%	89.000000	91.250000
max	92.000000	95.000000

```
In [40]: print(co2.describe())
```

	Engine Size(L)	Cylinders	Fuel Consumption City (L/100 km) \
count	7385.000000	7385.000000	7385.000000
mean	3.160068	5.615030	12.556534
std	1.354170	1.828307	3.500274
min	0.900000	3.000000	4.200000
25%	2.000000	4.000000	10.100000
50%	3.000000	6.000000	12.100000
75%	3.700000	6.000000	14.600000
max	8.400000	16.000000	30.600000

	Fuel Consumption Hwy (L/100 km)	Fuel Consumption Comb (L/100 km) \
count	7385.000000	7385.000000
mean	9.041706	10.975071
std	2.224456	2.892506
min	4.000000	4.100000
25%	7.500000	8.900000
50%	8.700000	10.600000
75%	10.200000	12.600000
max	20.600000	26.100000

	Fuel Consumption Comb (mpg)	CO2 Emissions(g/km)
count	7385.000000	7385.000000
mean	27.481652	250.584699
std	7.231879	58.512679
min	11.000000	96.000000
25%	22.000000	208.000000
50%	27.000000	246.000000
75%	32.000000	288.000000
max	69.000000	522.000000

```
In [41]: sales_data = {
    "Product": ["Laptop", "Mouse", "Keyboard", "Monitor"],
    "Quantity": [15, 40, 25, 12],
    "Price": [55000, 800, 1500, 12000]
}

sales = pd.DataFrame(sales_data)

print(sales.describe())
```

	Quantity	Price
count	4.000000	4.000000
mean	23.000000	17325.000000
std	12.622731	25633.750538
min	12.000000	800.000000
25%	14.250000	1325.000000
50%	20.000000	6750.000000
75%	28.750000	22750.000000
max	40.000000	55000.000000

Remove Duplicate Rows

```
In [42]: student_data = {
    "Name": ["Siya", "Aman", "Riya", "Aman", "Karan"],
    "Marks": [85, 92, 78, 92, 88],
    "Attendance": [90, 95, 82, 95, 89]
}
```

```
students2 = pd.DataFrame(student_data)

students2 = students2.drop_duplicates()

print(students2)
```

	Name	Marks	Attendance
0	Siya	85	90
1	Aman	92	95
2	Riya	78	82
4	Karan	88	89

```
In [43]: product_data = {
    "Product": ["Laptop", "Mouse", "Keyboard", "Mouse", "Monitor"],
    "Price": [55000, 800, 1500, 800, 12000],
    "Stock": [10, 50, 30, 50, 15]
}

products = pd.DataFrame(product_data)

products = products.drop_duplicates()

print(products)
```

	Product	Price	Stock
0	Laptop	55000	10
1	Mouse	800	50
2	Keyboard	1500	30
4	Monitor	12000	15

```
In [44]: employee_data = {
    "Employee_ID": [101, 102, 103, 102, 104],
    "Name": ["Aman", "Riya", "Karan", "Riya", "Neha"],
    "Department": ["IT", "HR", "Finance", "HR", "IT"]
}

employees2 = pd.DataFrame(employee_data)

employees2 = employees2.drop_duplicates()

print(employees2)
```

	Employee_ID	Name	Department
0	101	Aman	IT
1	102	Riya	HR
2	103	Karan	Finance
4	104	Neha	IT

Fill Missing Values

```
In [45]: student_data = {
    "Name": ["Siya", "Aman", "Riya", "Karan"],
    "Marks": [85, None, 78, 88],
    "Attendance": [90, 95, None, 89]
}

students3 = pd.DataFrame(student_data)

students3 = students3.fillna(0)
```

```
print(students3)
```

	Name	Marks	Attendance
0	Siya	85.0	90.0
1	Aman	0.0	95.0
2	Riya	78.0	0.0
3	Karan	88.0	89.0

```
In [46]: employee_data = {
    "Employee": ["Aman", "Riya", "Karan", "Neha"],
    "Salary": [45000, None, 52000, None],
    "Department": ["IT", "HR", "Finance", "IT"]
}

employees3 = pd.DataFrame(employee_data)

employees3["Salary"] = employees3["Salary"].fillna(0)

print(employees3)
```

	Employee	Salary	Department
0	Aman	45000.0	IT
1	Riya	0.0	HR
2	Karan	52000.0	Finance
3	Neha	0.0	IT

```
In [47]: product_data = {
    "Product": ["Laptop", "Mouse", "Keyboard", "Monitor"],
    "Stock": [10, None, 30, None],
    "Price": [55000, 800, 1500, 12000]
}

products3 = pd.DataFrame(product_data)

products3["Stock"] = products3["Stock"].fillna(0)

print(products3)
```

	Product	Stock	Price
0	Laptop	10.0	55000
1	Mouse	0.0	800
2	Keyboard	30.0	1500
3	Monitor	0.0	12000

Reset DataFrame Index

```
In [49]: student_data = {
    "Name": ["Siya", "Aman", "Riya", "Karan"],
    "Marks": [85, 92, 78, 88]
}

students4 = pd.DataFrame(student_data)

students4 = students4[students4["Marks"] >= 85]

print(students4)

students4 = students4.reset_index(drop=True)
```

```
print(students4)
```

	Name	Marks
0	Siya	85
1	Aman	92
3	Karan	88

	Name	Marks
0	Siya	85
1	Aman	92
2	Karan	88

```
In [50]: product_data = {
    "Product": ["Laptop", "Mouse", "Keyboard", "Monitor"],
    "Price": [55000, 800, 1500, 12000]
}

products4 = pd.DataFrame(product_data)

products4 = products4[products4["Price"] > 1000]

products4 = products4.reset_index(drop=True)

print(products4)
```

	Product	Price
0	Laptop	55000
1	Keyboard	1500
2	Monitor	12000

```
In [51]: employee_data = {
    "Employee": ["Aman", "Riya", "Karan", "Neha"],
    "Salary": [45000, 40000, 52000, 48000]
}

employees4 = pd.DataFrame(employee_data)

employees4 = employees4[employees4["Salary"] >= 45000]

employees4 = employees4.reset_index(drop=True)

print(employees4)
```

	Employee	Salary
0	Aman	45000
1	Karan	52000
2	Neha	48000

Remove Rows or Columns

```
In [52]: student_data = {
    "Name": ["Siya", "Aman", "Riya", "Karan"],
    "Marks": [85, 92, 78, 88],
    "Attendance": [90, 95, 82, 89]
}

students5 = pd.DataFrame(student_data)

students5 = students5.drop("Attendance", axis=1)
```

```
print(students5)
```

	Name	Marks
0	Siya	85
1	Aman	92
2	Riya	78
3	Karan	88

```
In [53]: product_data = {
    "Product": ["Laptop", "Mouse", "Keyboard", "Monitor"],
    "Price": [55000, 800, 1500, 12000],
    "Stock": [10, 50, 30, 15]
}

products5 = pd.DataFrame(product_data)

products5 = products5.drop("Stock", axis=1)

print(products5)
```

	Product	Price
0	Laptop	55000
1	Mouse	800
2	Keyboard	1500
3	Monitor	12000

```
In [54]: employee_data = {
    "Employee": ["Aman", "Riya", "Karan", "Neha"],
    "Department": ["IT", "HR", "Finance", "IT"],
    "Salary": [45000, 40000, 52000, 48000]
}

employees5 = pd.DataFrame(employee_data)

employees5 = employees5.drop(2, axis=0)

print(employees5)
```

	Employee	Department	Salary
0	Aman	IT	45000
1	Riya	HR	40000
3	Neha	IT	48000

Convert Pandas Data into Python List

```
In [55]: student_data = {
    "Name": ["Siya", "Aman", "Riya", "Karan"],
    "Marks": [85, 92, 78, 88]
}

students6 = pd.DataFrame(student_data)

names = students6["Name"].to_list()

print(names)
```

```
['Siya', 'Aman', 'Riya', 'Karan']
```

```
In [56]: product_data = {
    "Product": ["Laptop", "Mouse", "Keyboard", "Monitor"],
    "Price": [55000, 800, 1500, 12000]
}

products6 = pd.DataFrame(product_data)

prices = products6["Price"].to_list()

print(prices)
```

```
[55000, 800, 1500, 12000]
```

```
In [57]: employee_data = {
    "Employee": ["Aman", "Riya", "Karan", "Neha"],
    "Department": ["IT", "HR", "Finance", "IT"]
}

employees6 = pd.DataFrame(employee_data)

departments = employees6["Department"].to_list()

print(departments)
```

```
['IT', 'HR', 'Finance', 'IT']
```

Apply a Function to Data

```
In [58]: student_data = {
    "Name": ["Siya", "Aman", "Riya", "Karan"],
    "Marks": [85, 92, 78, 88]
}

students7 = pd.DataFrame(student_data)

def grade(marks):
    if marks >= 90:
        return "A"
    elif marks >= 80:
        return "B"
    else:
        return "C"

students7["Grade"] = students7["Marks"].apply(grade)

print(students7)
```

	Name	Marks	Grade
0	Siya	85	B
1	Aman	92	A
2	Riya	78	C
3	Karan	88	B

```
In [59]: employee_data = {
    "Employee": ["Aman", "Riya", "Karan", "Neha"],
    "Monthly_Salary": [45000, 40000, 52000, 48000]
}
```

```

employees7 = pd.DataFrame(employee_data)

def annual_salary(salary):
    return salary * 12

employees7["Annual_Salary"] = employees7["Monthly_Salary"].apply(annual_salary)

print(employees7)

```

	Employee	Monthly_Salary	Annual_Salary
0	Aman	45000	540000
1	Riya	40000	480000
2	Karan	52000	624000
3	Neha	48000	576000

```

In [60]: product_data = {
    "Product": ["Laptop", "Mouse", "Keyboard", "Monitor"],
    "Price": [55000, 800, 1500, 12000]
}

products7 = pd.DataFrame(product_data)

def discount(price):
    return price * 0.90

products7["Discounted_Price"] = products7["Price"].apply(discount)

print(products7)

```

	Product	Price	Discounted_Price
0	Laptop	55000	49500.0
1	Mouse	800	720.0
2	Keyboard	1500	1350.0
3	Monitor	12000	10800.0

Group Data for Analysis

```

In [61]: student_data = {
    "Name": ["Siya", "Aman", "Riya", "Karan", "Neha", "Rohit"],
    "Class": ["A", "A", "B", "B", "A", "B"],
    "Marks": [85, 92, 78, 88, 90, 75]
}

students8 = pd.DataFrame(student_data)

result = students8.groupby("Class")["Marks"].mean()

print(result)

```

```

Class
A    89.000000
B    80.333333
Name: Marks, dtype: float64

```

```

In [62]: employee_data = {
    "Employee": ["Aman", "Riya", "Karan", "Neha", "Rohit", "Priya"],
    "Department": ["IT", "HR", "IT", "Finance", "HR", "Finance"],
    "Salary": [45000, 40000, 52000, 48000, 42000, 55000]
}

```

```
employees8 = pd.DataFrame(employee_data)

result = employees8.groupby("Department")["Salary"].mean()

print(result)
```

```
Department
Finance    51500.0
HR         41000.0
IT         48500.0
Name: Salary, dtype: float64
```

```
In [ ]: sales_data = {
        "Product": ["Laptop", "Mouse", "Laptop", "Keyboard", "Mouse", "Laptop"],
        "Sales": [55000, 800, 60000, 1500, 1200, 50000]
    }

sales8 = pd.DataFrame(sales_data)

result = sales8.groupby("Product")["Sales"].sum()

print(result)
```

Calculate Total

```
In [63]: student_data = {
        "Name": ["Siya", "Aman", "Riya", "Karan"],
        "Marks": [85, 92, 78, 88]
    }

students9 = pd.DataFrame(student_data)

print(students9["Marks"].sum())
```

343

```
In [64]: sales_data = {
        "Product": ["Laptop", "Mouse", "Keyboard", "Monitor"],
        "Sales": [55000, 800, 1500, 12000]
    }

sales9 = pd.DataFrame(sales_data)

print(sales9["Sales"].sum())
```

69300

```
In [65]: employee_data = {
        "Employee": ["Aman", "Riya", "Karan", "Neha"],
        "Salary": [45000, 40000, 52000, 48000]
    }

employees9 = pd.DataFrame(employee_data)

print(employees9["Salary"].sum())
```

185000

Calculate Average

```
In [66]: student_data = {
          "Name": ["Siya", "Aman", "Riya", "Karan"],
          "Marks": [85, 92, 78, 88]
        }

students10 = pd.DataFrame(student_data)

print(students10["Marks"].mean())
```

85.75

```
In [67]: employee_data = {
          "Employee": ["Aman", "Riya", "Karan", "Neha"],
          "Salary": [45000, 40000, 52000, 48000]
        }

employees10 = pd.DataFrame(employee_data)

print(employees10["Salary"].mean())
```

46250.0

```
In [68]: product_data = {
          "Product": ["Laptop", "Mouse", "Keyboard", "Monitor"],
          "Price": [55000, 800, 1500, 12000]
        }

products10 = pd.DataFrame(product_data)

print(products10["Price"].mean())
```

17325.0

Find Index of Maximum Value

```
In [69]: student_data = {
          "Name": ["Siya", "Aman", "Riya", "Karan"],
          "Marks": [85, 92, 78, 88]
        }

students11 = pd.DataFrame(student_data)

index = students11["Marks"].idxmax()

print(index)
print(students11.loc[index])
```

1

```
Name      Aman
Marks      92
Name: 1, dtype: object
```

```
In [70]: employee_data = {
          "Employee": ["Aman", "Riya", "Karan", "Neha"],
          "Salary": [45000, 40000, 52000, 48000]
```

```

}

employees11 = pd.DataFrame(employee_data)

index = employees11["Salary"].idxmax()

print(index)
print(employees11.loc[index])

```

```

2
Employee    Karan
Salary      52000
Name: 2, dtype: object

```

```

In [71]: sales_data = {
        "Product": ["Laptop", "Mouse", "Keyboard", "Monitor"],
        "Sales": [55000, 800, 1500, 12000]
    }

sales11 = pd.DataFrame(sales_data)

index = sales11["Sales"].idxmax()

print(index)
print(sales11.loc[index])

```

```

0
Product    Laptop
Sales      55000
Name: 0, dtype: object

```

Data ko label/condition ke basis par select karna

```

In [72]: students = pd.DataFrame({
        "Name": ["Siya", "Aman", "Riya", "Karan"],
        "Marks": [85, 92, 78, 88],
        "Attendance": [90, 95, 82, 89]
    })

result = students.loc[students["Marks"] >= 85]

print(result)

```

	Name	Marks	Attendance
0	Siya	85	90
1	Aman	92	95
3	Karan	88	89

```

In [73]: employees = pd.DataFrame({
        "Name": ["Aman", "Riya", "Karan", "Neha"],
        "Department": ["IT", "HR", "IT", "Finance"],
        "Salary": [50000, 45000, 60000, 55000]
    })

result = employees.loc[:, ["Name", "Salary"]]

print(result)

```

	Name	Salary
0	Aman	50000
1	Riya	45000
2	Karan	60000
3	Neha	55000

```
In [74]: products = pd.DataFrame({
    "Product": ["Laptop", "Mouse", "Keyboard", "Monitor"],
    "Price": [55000, 800, 1500, 12000],
    "Stock": [10, 50, 30, 15]
})

result = products.loc[products["Price"] > 1000, ["Product", "Price"]]

print(result)
```

	Product	Price
0	Laptop	55000
2	Keyboard	1500
3	Monitor	12000

Multiple DataFrames ko combine karna

```
In [75]: class_a = pd.DataFrame({
    "Name": ["Siya", "Aman"],
    "Marks": [85, 92]
})

class_b = pd.DataFrame({
    "Name": ["Riya", "Karan"],
    "Marks": [78, 88]
})

result = pd.concat([class_a, class_b], ignore_index=True)

print(result)
```

	Name	Marks
0	Siya	85
1	Aman	92
2	Riya	78
3	Karan	88

```
In [76]: january = pd.DataFrame({
    "Product": ["Laptop", "Mouse"],
    "Sales": [10, 25]
})

february = pd.DataFrame({
    "Product": ["Laptop", "Mouse"],
    "Sales": [15, 30]
})

result = pd.concat([january, february], ignore_index=True)

print(result)
```

	Product	Sales
0	Laptop	10
1	Mouse	25
2	Laptop	15
3	Mouse	30

```
In [77]: it = pd.DataFrame({
        "Name": ["Aman", "Riya"],
        "Department": ["IT", "IT"],
        "Salary": [50000, 55000]
    })

    hr = pd.DataFrame({
        "Name": ["Karan", "Neha"],
        "Department": ["HR", "HR"],
        "Salary": [45000, 48000]
    })

    result = pd.concat([it, hr], ignore_index=True)

    print(result)
```

	Name	Department	Salary
0	Aman	IT	50000
1	Riya	IT	55000
2	Karan	HR	45000
3	Neha	HR	48000

```
import numpy as np
```

===== NUMPY IMPLEMENTATIONS =====

1. np.array()

Converts a Python list into a NumPy array.

2. type()

Checks the data type of an object.

3. .dtype

Shows the data type of the elements in the array.

4. .ndim

Shows the number of dimensions of the array (1D, 2D, 3D, etc.).

5. .size

Shows the total number of elements in the array.

6. .sum()

Calculates the total/sum of all elements.

7. .shape

Shows the structure of the array, such as rows and columns.

8. .reshape()

Changes the shape of an array without changing its data.

9. np.eye()

Creates an identity matrix with 1s on the main diagonal.

10. np.zeros()

Creates an array filled with zeros.

11. .std()

Calculates the standard deviation to measure data spread.

12. .var()

Calculates the variance to measure data spread.

13. .copy()

Creates an independent copy of the original array.

14. .sum(axis=0) / .sum(axis=1)

axis=0 calculates the sum column-wise.

axis=1 calculates the sum row-wise.

15. .ravel()

Flattens the array into 1D and generally returns a view of the original array.

16. .flatten()

Flattens the array into 1D and returns an independent copy.

17. Boolean Masking ($>$, $<$, $>=$, $<=$, $\&$, $\sim$)

Filters array elements based on one or more conditions.

18. `.transpose()`

Swaps the rows and columns of an array.

19. Broadcasting ($\%$, $+$, $-$, $*$, $/$)

Applies an element-wise operation across compatible arrays or values.

20. `np.where()`

Selects values based on whether a condition is True or False.

21. `np.select()`

Selects different values based on multiple conditions.

In []: